13. Sync Committee Service 구현 분석

개요

SyncCommitteeService는 Lighthouse 검증자 클라이언트의 핵심 서비스 중 하나로, Ethereum 2.0 Altair 포크에서 도입된 동기화 위원회(Sync Committee) 프로토콜을 구현합니다. 이 서비스는 검증자들이 동기화 위원회 멤버로서 수행해야 하는 듀티들을 자동화하고 관리합니다.

동기화 위원회는 라이트 클라이언트의 빠른 동기화를 지원하기 위해 도입된 메커니즘으로, 512명의 검증자가 각 슬롯마다 현재 헤드 블록에 대한 서명을 제공합니다. SyncCommitteeService는 다음과 같은 주요 기능을 제공합니다:

- 동기화 위원회 메시지(SyncCommitteeMessage) 생성 및 제출
- 동기화 위원회 기여(SyncCommitteeContribution) 집계 및 제출
- 서브넷 구독 관리
- 집계자 선택 및 증명 생성

아키텍처 및 구조

SyncCommitteeService 구조체

```
// validator_client/validator_services/src/sync_committee_service.rs

pub  struct SyncCommitteeService<S: ValidatorStore, T: SlotClock + 'static> {

inner: Arc<Inner<S, T>>,

}
```

```rust
pub struct Inner<S: ValidatorStore, T: SlotClock + 'static> {
duties_service: Arc<DutiesService<S, T>>,
validator_store: Arc<S>,
slot_clock: T,
beacon_nodes: Arc<BeaconNodeFallback<T>>,
executor: TaskExecutor,
/// 서비스가 최소 한 번 구독을 게시했는지 추적하는 불린 값
first_subscription_done: AtomicBool,
}
```

SyncCommitteeService는 다음과 같은 핵심 컴포넌트들과 상호작용합니다:

1. **DutiesService**: 동기화 위원회 듀티 정보 제공
2. **ValidatorStore**: 키 관리 및 서명 기능
3. **BeaconNodeFallback**: 비콘 노드와의 통신
4. **SlotClock**: 시간 동기화
5. **TaskExecutor**: 비동기 작업 실행

생성자 및 초기화

```rust
impl<S: ValidatorStore + 'static, T: SlotClock + 'static> SyncCommitteeService<S
pub fn new(
duties_service: Arc<DutiesService<S, T>>,
validator_store: Arc<S>,
slot_clock: T,
```

```
beacon_nodes: Arc<BeaconNodeFallback<T>>,
executor: TaskExecutor,
) -> Self {
Self {
inner: Arc::new(Inner {
duties_service,
validator_store,
slot_clock,
beacon_nodes,
executor,
first_subscription_done: AtomicBool::new(false),
}),
}
}
}
```

서비스 시작 및 스케줄링

start_update_service() 메서드

SyncCommitteeService는 `start_update_service()` 메서드를 통해 주기적인 동기화 위원회 작업을 시작합니다:

```
pub fn start_update_service(self, spec: &ChainSpec) -> Result<(), String> {
if self.duties_service.disable_attesting {
```

```
info!("Sync committee service disabled");
return Ok(());
}

let slot_duration = Duration::from_secs(spec.seconds_per_slot);
let duration_to_next_slot = self
.slot_clock
.duration_to_next_slot()
.ok_or("Unable to determine duration to next slot")?;

info!(
next_update_millis = duration_to_next_slot.as_millis(),
"Sync committee service started"
);

let executor = self.executor.clone();

let interval_fut = async move {
loop {
if let Some(duration_to_next_slot) = self.slot_clock.duration_to_next_slot() {
// 기여 브로드캐스트를 위해 슬롯의 1/3 지점에서 대기
sleep(duration_to_next_slot + slot_duration / 3).await;
```

```
// Altair 포크가 아직 발생하지 않았다면 아무것도 하지 않음
if !self.altair_fork_activated() {
continue;
}

if  let Err(e) = self.spawn_contribution_tasks(slot_duration).await {
crit!(
error = ?e,
"Failed to spawn sync contribution tasks"
)
} else {
trace!("Spawned sync contribution tasks")
}

// 미래 슬롯/에포크에 대한 구독 수행
self.spawn_subscription_tasks();
} else {
error!("Failed to read slot clock");
// 슬롯 클록을 읽을 수 없으면 다음 슬롯까지 대기
sleep(slot_duration).await;
}
}
};
```

```
executor.spawn(interval_fut, "sync_committee_service");
Ok(())
}
```

핵심 특징:

- 각 슬롯의 1/3 지점에서 기여 작업을 시작
- Altair 포크 활성화 여부를 확인하여 조건부 실행
- 슬롯 클록 동기화를 통한 정확한 타이밍 제어
- 에러 발생 시 로깅 및 복구 메커니즘

Altair 포크 활성화 확인

```
fn altair_fork_activated(&self) -> bool {
self.duties_service
.spec
.altair_fork_epoch
.and_then(|fork_epoch| {
let current_epoch = self.slot_clock.now()?.epoch(S::E::slots_per_epoch());
Some(current_epoch >= fork_epoch)
})
.unwrap_or(false)
}
```

이 메서드는 현재 에포크가 Altair 포크 에포크 이후인지 확인하여 동기화 위원회 기능의 활성화 여부를 결정합니다.

핵심 동기화 위원회 프로세스

spawn_contribution_tasks() 메서드

이 메서드는 현재 슬롯에 대한 모든 동기화 위원회 작업을 스폰합니다:

```
async fn spawn_contribution_tasks(&self, slot_duration: Duration) -> Result<()

let slot = self.slot_clock.now().ok_or("Failed to read slot clock")?;

let duration_to_next_slot = self

.slot_clock

.duration_to_next_slot()

.ok_or("Unable to determine duration to next slot")?;


// 검증자가 동기화 집계를 게시해야 하는 경우, 슬롯의 2/3 지점에서 수행해야 함

let aggregate_production_instant = Instant::now()

+ duration_to_next_slot

.checked_sub(slot_duration / 3)

.unwrap_or_else(|| Duration::from_secs(0));


let Some(slot_duties) = self

.duties_service

.sync_duties
```

```
.get_duties_for_slot::<S::E>(slot, &self.duties_service.spec)
else {
debug!("No duties known for slot {}", slot);
return Ok(());
};


if slot_duties.duties.is_empty() {
debug!(%slot, "No local validators in current sync committee");
return Ok(());
}


// 비관적이지 않은 실행을 위해 `block_root` 가져오기
let response = self
.beacon_nodes
.first_success(
|beacon_node| async move {
match beacon_node.get_beacon_blocks_root(BlockId::Head).await {
Ok(Some(block)) if block.execution_optimistic == Some(false) => {
Ok(block)
}
Ok(Some(_)) => {
Err(format!("To sign sync committee messages for slot {slot} a non-optimistic he
}
```

```
Ok(None) => Err(format!("No block root found for slot {}", slot)),
Err(e) => Err(e.to_string()),
}
},
)
.await;

let  block_root = match  response {
Ok(block) => block.data.root,
Err(errs) => {
warn!(
errors = errs.to_string(),
%slot,
"Refusing to sign sync committee messages for an optimistic head block or \
a block head with unknown optimistic status"
);
return Ok(());
}
};

// 모든 동기화 위원회 서명을 게시하는 하나의 작업 스폰
let  validator_duties = slot_duties.duties;
let  service = self.clone();
```

```
self.inner.executor.spawn(
async  move {
service
.publish_sync_committee_signatures(slot, block_root, validator_duties)
.map(|_| ())
.await
},
"sync_committee_signature_publish",
);


let  aggregators = slot_duties.aggregators;
let  service = self.clone();
self.inner.executor.spawn(
async  move {
service
.publish_sync_committee_aggregates(
slot,
block_root,
aggregators,
aggregate_production_instant,
)
.map(|_| ())
.await
```

```
},
"sync_committee_aggregate_publish",
);

Ok(())
}
```

핵심 특징:

- 비관적이지 않은(non-optimistic) 헤드 블록 루트 확인
- 개별 서명과 집계 작업을 별도의 비동기 작업으로 분리
- 집계 생성을 위한 정확한 타이밍 계산 (슬롯의 2/3 지점)

동기화 위원회 메시지 생성 및 제출

publish_sync_committee_signatures() 메서드

이 메서드는 개별 동기화 위원회 메시지의 전체 생명주기를 관리합니다:

```
async fn publish_sync_committee_signatures(
&self,
slot: Slot,
beacon_block_root: Hash256,
validator_duties: Vec<SyncDuty>,
) -> Result<(), ()> {
// 동기화 위원회 서명을 생성하는 퓨처들 생성
```

```
let signature_futures = validator_duties.iter().map(|duty| async move {
match self
.validator_store
.produce_sync_committee_signature(
slot,
beacon_block_root,
duty.validator_index,
&duty.pubkey,
)
.await
{
Ok(signature) => Some(signature),
Err(ValidatorStoreError::UnknownPubkey(pubkey)) => {
// 검증자가 최근에 API를 통해 제거된 경우 pubkey가 누락될 수 있음
debug!(
?pubkey,
validator_index = duty.validator_index,
%slot,
"Missing pubkey for sync committee signature"
);
None
}
Err(e) => {
```

```
crit!(
validator_index = duty.validator_index,
%slot,
error = ?e,
"Failed to sign sync committee signature"
);
None
}
}
});


// 모든 퓨처를 병렬로 실행하여 성공한 결과들을 수집
let  committee_signatures = &join_all(signature_futures)
.await
.into_iter()
.flatten()
.collect::<Vec<_>>();


self.beacon_nodes
.request(ApiTopic::SyncCommittee, |beacon_node| async  move {
beacon_node
.post_beacon_pool_sync_committee_signatures(committee_signatures)
.await
```

```
})
.await
.map_err(|e| {
error!(
%slot,
error = %e,
"Unable to publish sync committee messages"
);
})?;

info!(
count = committee_signatures.len(),
head_block = ?beacon_block_root,
%slot,
"Successfully published sync committee messages"
);

Ok(())
}
```

핵심 특징:

- **병렬 서명 처리**: `join_all()` 을 사용하여 모든 검증자의 서명을 동시에 처리
- **에러 처리**: 각 검증자별 독립적인 에러 처리

- **배치 제출**: 여러 서명을 한 번의 HTTP 요청으로 제출

SyncCommitteeMessage 데이터 구조

동기화 위원회 메시지는 다음과 같은 구조를 가집니다:

```
// consensus/types/src/sync_committee_message.rs
pub struct SyncCommitteeMessage {
pub slot: Slot,
pub beacon_block_root: Hash256,
pub validator_index: u64,
// 검증자가 beacon_block_root에 대한 서명
pub signature: Signature,
}
```

동기화 위원회 집계 처리

publish_sync_committee_aggregates() 메서드

집계 처리는 서브넷별로 분리되어 수행됩니다:

```
async fn publish_sync_committee_aggregates(
&self,
slot: Slot,
beacon_block_root: Hash256,
aggregators: HashMap<SyncSubnetId, Vec<(u64, PublicKeyBytes, SyncSelectionProof)
```

```
aggregate_instant: Instant,
) {
for (subnet_id, subnet_aggregators) in  aggregators {
let  service = self.clone();
self.inner.executor.spawn(
async  move {
service
.publish_sync_committee_aggregate_for_subnet(
slot,
beacon_block_root,
subnet_id,
subnet_aggregators,
aggregate_instant,
)
.map(|_| ())
.await
},
"sync_committee_aggregate_publish_subnet",
);
}
}
```

publish_sync_committee_aggregate_for_subnet() 메서드

각 서브넷별 집계 처리의 상세 구현:

```
async fn publish_sync_committee_aggregate_for_subnet(
&self,
slot: Slot,
beacon_block_root: Hash256,
subnet_id: SyncSubnetId,
subnet_aggregators: Vec<(u64, PublicKeyBytes, SyncSelectionProof)>,
aggregate_instant: Instant,
) -> Result<(), ()> {
sleep_until(aggregate_instant).await;


let contribution = &self
.beacon_nodes
.first_success(|beacon_node| async move {
let sync_contribution_data = SyncContributionData {
slot,
beacon_block_root,
subcommittee_index: subnet_id.into(),
};


beacon_node
.get_validator_sync_committee_contribution(&sync_contribution_data)
```

```
.await
})
.await
.map_err(|e| {
crit!(
%slot,
?beacon_block_root,
error = %e,
"Failed to produce sync contribution"
)
})?
.ok_or_else(|| {
crit!(%slot, ?beacon_block_root, "No aggregate contribution found");
})?
.data;


// 서명된 기여를 생성하는 퓨처들 생성
let  signature_futures = subnet_aggregators.into_iter().map(
|(aggregator_index, aggregator_pk, selection_proof)| async  move {
match  self
.validator_store
.produce_signed_contribution_and_proof(
aggregator_index,
```

```
aggregator_pk,
contribution.clone(),
selection_proof,
)
.await
{
Ok(signed_contribution) => Some(signed_contribution),
Err(ValidatorStoreError::UnknownPubkey(pubkey)) => {
// 검증자가 최근에 API를 통해 제거된 경우 pubkey가 누락될 수 있음
debug!(?pubkey, %slot, "Missing pubkey for sync contribution");
None
}
Err(e) => {
crit!(
%slot,
error = ?e,
"Unable to sign sync committee contribution"
);
None
}
}
},
);
```

```
// 모든 퓨처를 병렬로 실행하여 성공한 결과들을 수집
let  signed_contributions = &join_all(signature_futures)
.await
.into_iter()
.flatten()
.collect::<Vec<_>>();


// 비콘 노드에 게시
self.beacon_nodes
.first_success(|beacon_node| async  move {
beacon_node
.post_validator_contribution_and_proofs(signed_contributions)
.await
})
.await
.map_err(|e| {
error!(
%slot,
error = %e,
"Unable to publish signed contributions and proofs"
);
})?;
```

```
info!(
subnet = %subnet_id,
beacon_block_root = %beacon_block_root,
num_signers = contribution.aggregation_bits.num_set_bits(),
%slot,
"Successfully published sync contributions"
);

Ok(())
}
```

집계 처리의 특징:

- **타이밍 제어**: `sleep_until()` 을 사용하여 정확한 집계 시점까지 대기
- **서브넷별 처리**: 각 동기화 서브넷별로 독립적인 집계 처리
- **기여 다운로드**: 비콘 노드에서 집계된 기여를 다운로드
- **서명된 증명 생성**: 집계자들의 서명된 기여 및 증명 생성

SyncCommitteeContribution 데이터 구조

동기화 위원회 기여는 다음과 같은 구조를 가집니다:

```
// consensus/types/src/sync_committee_contribution.rs
pub struct SyncCommitteeContribution<E: EthSpec> {
pub slot: Slot,
```

```
pub beacon_block_root: Hash256,
pub subcommittee_index: u64,
pub aggregation_bits: BitVector<E::SyncSubcommitteeSize>,
pub signature: AggregateSignature,
}
```

서브넷 구독 관리

spawn_subscription_tasks() 메서드

동기화 위원회 서브넷 구독을 관리합니다:

```
fn spawn_subscription_tasks(&self) {
let service = self.clone();

self.inner.executor.spawn(
async move {
service.publish_subscriptions().await.unwrap_or_else(|e| {
error!(
error = ?e,
"Error publishing subscriptions"
)
});
},
```

```
"sync_committee_subscription_publish",
);
}
```

publish_subscriptions() 메서드

실제 구독 게시 로직:

```
async fn publish_subscriptions(self) -> Result<(), String> {
let spec = &self.duties_service.spec;
let slot = self.slot_clock.now().ok_or("Failed to read slot clock")?;

let mut duty_slots = vec![];
let mut all_succeeded = true;

// 현재 기간 동안 매 에포크 시작 시 비콘 노드에 구독을 재게시
// 이는 BN이 재시작으로 인해 구독을 잊어버린 경우나 VC가 대체 BN으로 전환한 경우를 다룸
let current_period = sync_period_of_slot::<S::E>(slot, spec)?;

if !self.first_subscription_done.load(Ordering::Relaxed)
|| slot.as_u64() % S::E::slots_per_epoch() == 0
{
duty_slots.push((slot, current_period));
}
```

```
// 현재 기간 말미에 다음 기간에 대한 구독을 비콘 노드에 푸시

// BN이 (충분히 미리) 구독되도록 보장하는 주요 방법으로 적극적으로 푸시

let lookahead_slot = slot + SUBSCRIPTION_LOOKAHEAD_EPOCHS * S::E::slots_per_epo


let lookahead_period = sync_period_of_slot::<S::E>(lookahead_slot, spec)?;


if lookahead_period > current_period {

duty_slots.push((lookahead_slot, lookahead_period));

}


if duty_slots.is_empty() {

return Ok(());

}


// 구독 수집

let mut subscriptions = vec![];


for (duty_slot, sync_committee_period) in duty_slots {

debug!(%duty_slot, %slot, "Fetching subscription duties");

match self

.duties_service

.sync_duties
```

```
.get_duties_for_slot::<S::E>(duty_slot, spec)
{
Some(duties) => subscriptions.extend(subscriptions_from_sync_duties(
duties.duties,
sync_committee_period,
spec,
)),
None => {
debug!(
slot = %duty_slot,
"No duties for subscription"
);
all_succeeded = false;
}
}
}

if  subscriptions.is_empty() {
debug!(%slot, "No sync subscriptions to send");
return Ok(());
}

// BN에 구독 게시
```

```
debug!(
count = subscriptions.len(),
"Posting sync subscriptions to BN"
);
let  subscriptions_slice = &subscriptions;


for  subscription  in  subscriptions_slice {
debug!(
validator_index = subscription.validator_index,
validator_sync_committee_indices = ?subscription.sync_committee_indices,
until_epoch = %subscription.until_epoch,
"Subscription"
);
}


if  let Err(e) = self
.beacon_nodes
.request(ApiTopic::Subscriptions, |beacon_node| async  move {
beacon_node
.post_validator_sync_committee_subscriptions(subscriptions_slice)
.await
})
.await
```

```
{
error!(
%slot,
error = %e,
"Unable to post sync committee subscriptions"
);
all_succeeded = false;
}

// 모든 듀티가 한 번 성공하면 첫 번째 구독 래치 비활성화
if  all_succeeded {
self.first_subscription_done.store(true, Ordering::Relaxed);
}

Ok(())
}
```

구독 관리의 특징:

- **주기적 재구독**: 매 에포크마다 구독 상태 갱신
- **미리 보기 구독**: 다음 동기화 위원회 기간에 대한 사전 구독
- **고가용성**: 비콘 노드 재시작이나 전환 시에도 구독 유지

SyncCommitteeSubscription 데이터 구조

```rust
// consensus/types/src/sync_committee_subscription.rs
pub struct SyncCommitteeSubscription {
/// 검증자 인덱스
pub validator_index: u64,
/// 동기화 위원회 내에서의 검증자 인덱스들
pub sync_committee_indices: Vec<u64>,
/// 구독이 유효한 마지막 에포크
pub until_epoch: Epoch,
}
```

동기화 위원회 듀티 관리

SyncDuty 데이터 구조

```rust
// consensus/types/src/sync_duty.rs
pub struct SyncDuty {
pub pubkey: PublicKeyBytes,
pub validator_index: u64,
pub validator_sync_committee_indices: Vec<u64>,
}

impl SyncDuty {
/// 동기화 위원회 인덱스 목록에서 새로운 `SyncDuty` 생성
```

```
pub fn from_sync_committee_indices(
validator_index: u64,
pubkey: PublicKeyBytes,
sync_committee_indices: &[usize],
) -> Option<Self> {
// 위원회 내에서 `validator_index`의 위치들
let validator_sync_committee_indices = sync_committee_indices
.iter()
.enumerate()
.filter_map(|(i, &v)| {
if validator_index == v as u64 {
Some(i as u64)
} else {
None
}
})
.collect();
Self::new(validator_index, pubkey, validator_sync_committee_indices)
}


/// 이 듀티에 대한 서브넷 ID 집합 가져오기
pub fn subnet_ids<E: EthSpec>(&self) -> Result<HashSet<SyncSubnetId>, ArithErr
SyncSubnetId::compute_subnets_for_sync_committee::<E>(
```

```
&self.validator_sync_committee_indices,
)
}
}
```

동기화 위원회 기간 계산

```
fn sync_period_of_slot<E: EthSpec>(slot: Slot, spec: &ChainSpec) -> Result<u64,
slot.epoch(E::slots_per_epoch())
.sync_committee_period(spec)
.map_err(|e| format!("Error computing sync period: {:?}", e))
}

fn subscriptions_from_sync_duties(
duties: Vec<SyncDuty>,
sync_committee_period: u64,
spec: &ChainSpec,
) -> impl Iterator<Item = SyncCommitteeSubscription> {
let until_epoch = spec.epochs_per_sync_committee_period * (sync_committee_perio
duties
.into_iter()
.map(move |duty| SyncCommitteeSubscription {
validator_index: duty.validator_index,
```

```
sync_committee_indices: duty.validator_sync_committee_indices,

until_epoch,

})

}
```

성능 최적화 및 특징

1. 병렬 처리

- 모든 검증자의 서명을 `join_all()` 로 병렬 처리
- 서브넷별 독립적인 집계 작업 실행

2. 타이밍 최적화

- 슬롯의 1/3 지점에서 개별 메시지 처리 시작
- 슬롯의 2/3 지점에서 집계 처리 수행
- 정확한 타이밍을 위한 `sleep_until()` 사용

3. 네트워크 효율성

- 배치 제출을 통한 네트워크 오버헤드 최소화
- 서브넷별 분산 처리로 네트워크 부하 분산

4. 고가용성

- BeaconNodeFallback을 통한 자동 장애 조치
- 주기적 구독 갱신으로 연결 안정성 보장

에러 처리 및 복구

1. 검증자별 독립적 처리

- 한 검증자의 실패가 다른 검증자에게 영향을 주지 않음
- 각 검증자별 상세한 에러 로깅

2. 네트워크 장애 대응

- BeaconNodeFallback을 통한 자동 장애 조치
- 재시도 메커니즘 내장

3. 포크 호환성

- Altair 포크 활성화 여부 확인
- 포크 이전에는 동기화 위원회 작업 비활성화

실제 사용 예시

SyncCommitteeService의 전체 워크플로우는 다음과 같습니다:

1. **서비스 시작**: `start_update_service()` 호출
2. **포크 확인**: Altair 포크 활성화 여부 확인
3. **슬롯 동기화**: 각 슬롯의 1/3 지점에서 작업 시작
4. **듀티 수집**: DutiesService에서 현재 슬롯의 동기화 위원회 듀티 조회
5. **헤드 블록 확인**: 비관적이지 않은 헤드 블록 루트 확인
6. **개별 메시지 생성**:

- 각 검증자별 병렬 서명 처리

- 배치로 동기화 위원회 메시지 제출

7. **집계 처리**:

- 슬롯의 2/3 지점까지 대기
- 서브넷별 집계 기여 다운로드
- 집계자들의 SignedContributionAndProof 생성 및 제출

8. **구독 관리**: 동기화 위원회 서브넷 구독 갱신

이러한 체계적인 프로세스를 통해 SyncCommitteeService는 Ethereum 2.0 네트워크에서 라이트 클라이언트의 빠른 동기화를 지원하는 동기화 위원회 프로토콜을 안전하고 효율적으로 구현합니다.